GenAI Cheat Sheet

A Step-by-Step Learning Path for Developers & Professionals

LLMs → Prompting → Production → RAG → Agents



Foundation
Understand the core concepts

Communication
Master prompting techniques

Scale
Think like an engineer

Data Access
Leverage your data with RAG

Autonomy
Build agents that take action

LLMs → Prompting → Production → RAG → Agents

From Fundamentals to Production-Ready AI Systems

PROITBRIDGE
Strive For Better Future



AI Coach John
Follow for more
Learn. Build. Ship. Repeat.

PROITBRIDGE
Strive For Better Future

AI Coach John | Generative AI Fundamentals

Understand LLMs

The Foundation Everything Else Builds On

WHAT IS GENAI?

Generative AI is a class of AI systems that can generate new content — text, code, images, audio — by learning patterns from massive datasets.

Unlike traditional AI that classifies or predicts, GenAI creates.

Traditional AI
- → classify
- → predict
- → analyze

Generative AI
- → create
- → generate
- → synthesize

WHAT IS AN LLM?

A Large Language Model is a deep **neural network** trained on trillions **of tokens** of text.

It learns to predict the next token given a context.

At scale, this simple objective produces **emergent abilities: reasoning, coding, translation, summarization, and more.**

HOW LLMS GENERATE TEXT

Input Prompt
You provide a question or instruction.

Tokenization
Text is split into tokens (subwords).

Probability Distribution
Model analyzes context and assigns probabilities to all possible next tokens.

Next Token Prediction
Model selects the next token (highest probability or sampled).

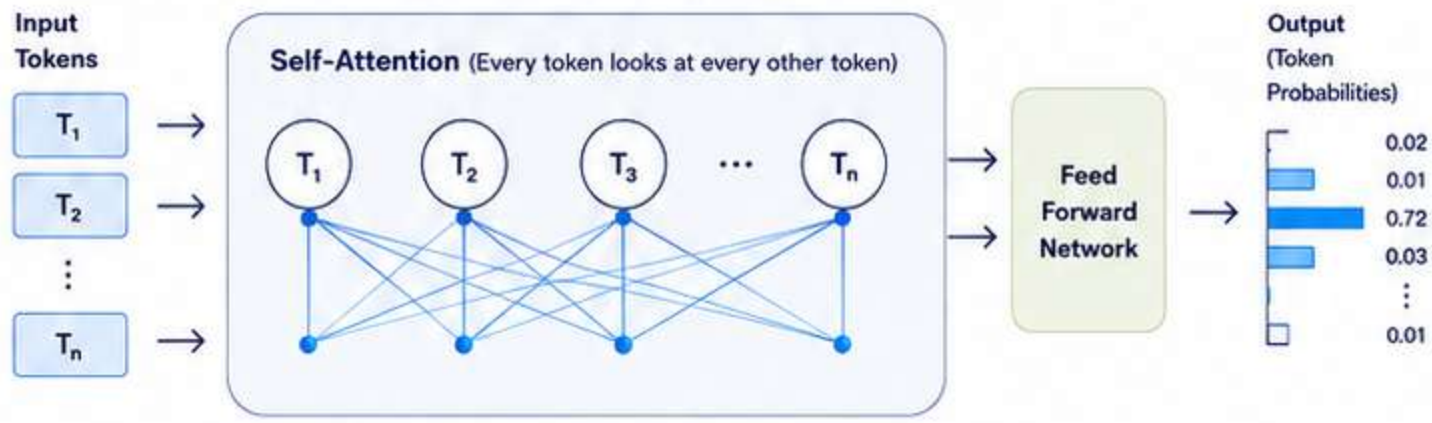
Generated Response
Tokens are repeated until the response is complete.

One token at a time.
One probability at a time.

LLM CORE MECHANISM

Context (Input Text) → Token Prediction → Next Token → Sequence Generation

TRANSFORMER MODEL (Simplified)



Key Idea: Attention lets every token decide what matters.

5 CORE CONCEPTS TO KNOW

1 TOKENS

The atomic unit of text an LLM reads and writes. Not words — subwords.

"unhappiness"
↓
["un", "happiness"]

- 1 token ≈ 0.75 words
- GPT-4 charges by token
- Context window is measured in tokens

2 CONTEXT WINDOW

The maximum number of tokens the model can see at once — both input and output combined.

Examples:

GPT-3	4K
GPT-4o	128K
Gemini 1.5 Pro	1M

Everything outside the window is invisible to the model.

3 TEMPERATURE

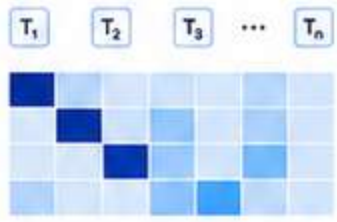
Controls randonmess of output.

- **0** — **Deterministic** Always picks the most probable token.
- **1** — **Natural Sampling** Balanced, more human-like output.
- **>1** — **Creative / Chaotic** More random, more unexpected.

Use low temperature for facts, high for creativity.

4 TRANSFORMER BASICS

The architecture behind every modern LLM.



- **Attention:** every token looks at every other token.
- Replaced RNNs in 2017.
- Enables massive parallelism during training.

5 TRAINING vs INFERENCE

Two very different phases.

TRAINING (Learning Phase)
- Trillions of tokens
- Weeks or months
- Thousands of GPUs
- Costs millions

INFERENCE (Usage Phase)
- Uses what it learned
- Millisecends
- Fractions of a cent
- You interact here

MODELS TO KNOW

Model	Creator	Type	Highlight
GPT-4o	OpenAI	Proprietary	Multimodal
Claude 3.5	Anthropic	Proprietary	Strong reasoning
Gemini 1.5	Google	Proprietary	1M context
LLaMA 3	Meta	Open-source	Run locally
Mistral	Mistral AI	Open-source	Efficient
Qwen 2.5	Alibaba	Open-source	Multilingual

HOW MODELS GENERATE TEXT (Step-by-Step)



1 **Prompt** You provide input.

2 **Tokenization** Text is split into tokens.

3 **Probability Scores** Model evaluates all possible next tokens.

4 **Token Selection** Chooses highest probability or samples based on temperature.

5 **Next Token** That token is added to the sequence.

Repeat until completion

GOAL: Understand how models generate text — one token at a time, one probability at a time.

PROITBRIDGE
Strive For Better Future

AI Coach John | Generative AI Fundamentals

Prompt Engineering – The Most Underrated Developer Skill

THE CORE IDEA

The model's output quality is directly proportional to the quality of your input.

Same model → Same question → Different prompt → Completely different result

Prompt engineering is not a hack. It's a discipline.

CORE TECHNIQUES

1 ZERO-SHOT PROMPTING

Give the task directly with no examples. Works when the task is well-defined and the model has strong priors.

"Classify this email as spam or not spam: [email]"

2 FEW-SHOT PROMPTING

Provide 2–5 input-output examples before your real query. The model learns the pattern from examples.

```
Input: "I love this!"   → Sentiment: Positive
Input: "This is awful"  → Sentiment: Negative
Input: "It's okay"      → Sentiment: ?
```

3 CHAIN-OF-THOUGHT (CoT)

Tell the model to think step by step before answering. Dramatically improves math, logic, and reasoning tasks.

"Solve this step by step: If a train travels 60 mph for 2.5 hours..."

4 ROLE PROMPTING

Assign the model a persona. Changes tone, depth, and framing of the response.

"You are a senior security engineer at a fintech company. Review this code for vulnerabilities."

5 STRUCTURED OUTPUT

Ask for a specific format — JSON, markdown table, numbered list, XML. Essential for programmatic use.

"Return your answer as a JSON object with keys: summary, sentiment, confidence_score"

6 PROMPT TEMPLATES

Build reusable prompt skeletons with variables. Separate the structure from the dynamic inputs.

```
"You are a {role}. Given the following {input_type}, {task}.
Format: {format}."
```

BAD vs BETTER PROMPT

BAD PROMPT:

"Summarize this."

Problems:
- No audience defined.
- No length.
- No format.
- No tone.

Result: Random length, random style, may miss key points.

BETTER PROMPT:

"Summarize the following article in exactly 5 bullet points for a C-suite executive audience. Focus on business impact, risks, and recommended actions. Be direct and avoid jargon."

Why it works:
- ✓ Audience defined.
- ✓ Format specified.
- ✓ Focus specified.
- ✓ Tone specified.
- ✓ Length controlled.
- ✓ Result is predictable.

PROMPT ANATOMY

[ROLE]	→	Who the model is
[CONTEXT]	→	Background information
[INSTRUCTION]	→	The actual task
[FORMAT]	→	How to structure the output
[CONSTRAINTS]	→	What NOT to do
[EXAMPLES]	→	Optional few-shot demonstrations

Better prompts = better outputs.
You control quality before the model ever runs.

Production LLM Thinking

— Where Most Projects Break Down

THE HARD TRUTH

Demo AI → 1 user, 1 prompt, works great in a notebook

Production AI → 10,000 users, 1,000 prompt variants, cost constraints, latency SLAs, failure modes

Most teams ship the demo and wonder why production is painful. These are the concepts that separate engineers who build toy apps from engineers who build real systems.

CORE CONCEPTS

1 PROMPT VERSIONING

Every prompt change is a code change.
Version your prompts in Git.
A/B test prompt versions.
Track which version produced which output.
Never change a production prompt without a rollback plan.

```
prompts/
├── summarize_v1.txt   ← previous
├── summarize_v2.txt   ← current (deployed)
└── summarize_v3.txt   ← testing
```

2 CACHING

If 40% of your users ask the same question, why call the LLM 40% of the time?
Semantic caching matches similar queries and returns cached responses.
Saves cost and cuts latency dramatically.

Exact cache:
"What is the capital of France?"
→ always Paris

Semantic cache:
"France capital?"
≈ "What's the capital of France?"

3 TOKEN COST OPTIMIZATION

Every token costs money.
Input tokens + Output tokens = bill.

- Trim whitespace and redundant instructions from prompts
- Use cheaper models for simple tasks (GPT-4o-mini vs GPT-4o)
- Use streaming to improve perceived performance
- Set max_tokens to limit runaway output

4 LATENCY AWARENESS

LLMs are slow by software standards. Time to First Token (TTFT) feels like forever at 3 seconds.

- Stream output token by token — feels 10× faster to the user
- Run non-dependent LLM calls in parallel
- Cache at multiple layers
- Use smaller models for latency-sensitive paths

Streaming makes 3s feel like 300ms

5 CONTEXT MANAGEMENT

Context windows are finite and expensive. Sending 100K tokens every call costs 25× more than 4K.

- Summarize old conversation turns instead of keeping all history
- Only inject the relevant retrieved chunks, not entire documents
- Use sliding window memory for long conversations

Long History (100K tokens) → Summarize → Relevant Context (4K tokens)

6 MONITORING

You cannot improve what you cannot see.

Track:

- Input tokens
- Output tokens
- Latency (TTFT + Total)
- Error rate
- Cost per query
- Guardrail triggers
- User feedback signals

Measure → Understand → Improve

THE PRODUCTION CHECKLIST

- [ ] Prompts versioned and tested before deploy
- [ ] Semantic cache in place for repeated queries
- [ ] Streaming enabled for user-facing responses
- [ ] Token budget set per request
- [ ] Retry logic with exponential backoff
- [ ] Fallback model configured (primary fails → secondary)
- [ ] Logging pipeline capturing inputs + outputs
- [ ] Cost alerting set up

GOAL

Build systems that scale, cost predictably, and don't break at 3am.



LLM Security: The Part Developers Skip Until It's Too Late



1 WHY THIS MATTERS NOW

Large Language Models introduce new security risks.
Traditional validation methods are not enough.
AI systems can be influenced through prompts, context, and user interactions.
Teams building GenAI products must design for security from the beginning.



2 SECURITY RISKS TO UNDERSTAND

1 PROMPT INJECTION

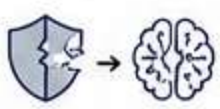


User instructions may attempt to override intended AI behavior and manipulate outputs.

Example:
"Ignore previous instructions" style prompt manipulation.

Defense:
- Input validation
- Output verification
- Permission separation

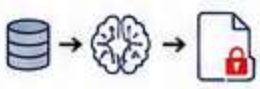
2 JAILBREAKING



Creative prompting techniques attempt to bypass model safeguards using indirect wording, fictional framing, or encoded instructions.

Defense:
- Safety classifiers
- Content moderation
- Guardrail systems

3 DATA LEAKAGE



AI systems may accidentally expose hidden instructions, retrieved context, or sensitive internal information.

Defense:
- Avoid storing secrets in prompts
- Use secure storage
- Limit exposed context

4 PII PROTECTION



Sensitive personal information may appear in prompts, outputs, or logs.

Defense:
- Detect and redact sensitive data
- Clean logs
- Use PII detection libraries

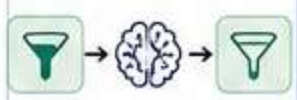
5 ACCESS CONTROL



Users should only access authorized information inside RAG pipelines and AI systems.

Defense:
- User scoped retrieval
- Permission filtering
- Organization level access checks

6 SAFETY GUARDRAILS



Additional validation layers run before and after model responses to reduce unsafe or unintended outputs.

Defense:
- Input filtering
- Output filtering
- Monitoring

3 DEFENSE IN DEPTH FLOW

USER INPUT
User sends a prompt or request to the system.

↓

INPUT CLASSIFIER
Detect unsafe prompts, abuse attempts, and sensitive data.

↓

PII REDACTION LAYER
Detect and redact PII and sensitive information.

↓

LLM PROCESSING
Prompt is sent to the model for processing.

↓

OUTPUT VALIDATION LAYER
Check for policy violations, toxicity, and unsafe content.

↓

PII CLEANUP LAYER
Remove or mask any sensitive information in the output.

↓

FINAL USER RESPONSE
Safe, validated, and cleaned response delivered to user.

4 KEY QUESTION



Can a malicious interaction manipulate your AI system, expose sensitive information, or affect other users?

If the answer is unclear, security testing should happen before deployment.





GOAL

Build GenAI systems that are secure by design, not added later as an afterthought.



Hallucination & Evaluation — You Can't Trust What You Can't Measure

THE CORE PROBLEM

Confident answer ≠ Correct answer

LLMs don't know what they don't know.
They generate the most statistically likely next token — not the most factually accurate one.

A model can cite a paper that doesn't exist, give you a wrong API endpoint with complete confidence, or invent a legal case and quote it perfectly formatted.



WHAT IS HALLUCINATION?

Hallucination is when an LLM generates content that is factually incorrect, fabricated, or not supported by any provided context — but states it confidently as fact.

TYPES

Factual Hallucination
Wrong facts stated as truth.
"Einstein won the Nobel Prize for Relativity"

Source Hallucination
Cites references that don't exist.

Context Hallucination
Contradicts the document you provided (RAG failure).

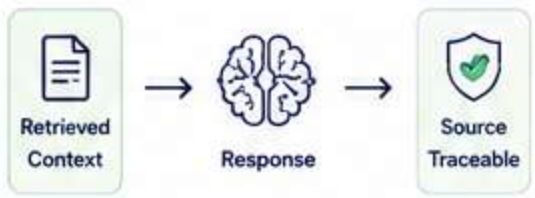
Instruction Hallucination
Claims to have done something it didn't.

CORE EVALUATION CONCEPTS

1 GROUNDED RESPONSES

Every claim in the response should trace back to a source — the retrieved context, a document, or verified knowledge.

If you can't point to where a fact came from, it's a hallucination risk.



2 LLM EVALUATION (LLM-as-Judge)

Use a stronger model (GPT-4o, Claude) to evaluate another model's output.

Score on correctness, relevance, faithfulness, format compliance. Scalable, cheap, and surprisingly good.

Judge prompt:
"Rate the following response 1-5 for factual accuracy.
Reference answer: [gold].
Model response: [output].
Score + reason:"

3 HUMAN EVALUATION

The gold standard. Humans rate outputs on a rubric.

Expensive and slow, but required for calibrating automated evals and final validation before launch.

RUBRIC EXAMPLE

1. Poor
2. Fair
3. Good
4. Very Good
5. Excellent

4 ACCURACY CHECKS

Programmatic assertions on model output:

```
assert "disclaimer" in response.lower()
# must include disclaimer

assert len(response.split()) < 200
# must be concise

assert json.loads(response)["score"] >= 0
# must be valid JSON
```

5 BENCHMARKING

Standard test suites to compare models:

- **MMLU** — 57 academic subjects, tests breadth of knowledge
- **HumanEval** — Code generation correctness
- **MT-Bench** — Multi-turn conversation quality
- **RAGAS** — RAG-specific: faithfulness, context recall, answer relevance

6 EVALUATION DIMENSIONS

Correctness	→	Is the answer factually right?
Faithfulness	→	Is it grounded in provided context?
Relevance	→	Does it answer the actual question?
Completeness	→	Did it address all parts of the query?
Conciseness	→	Is it free of unnecessary verbosity?
Format	→	Did it follow the output format requested?
Safety	→	Does it contain harmful or biased content?

GOAL

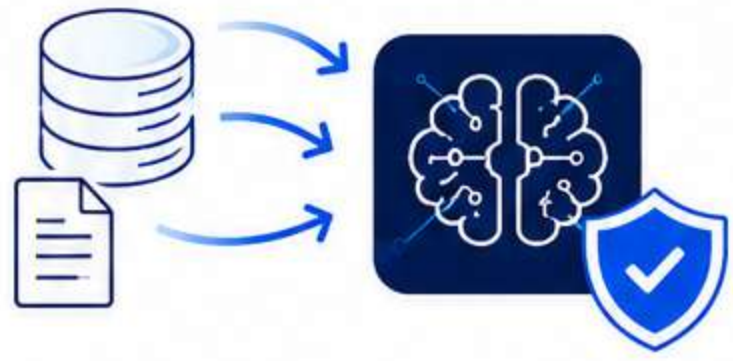
Don't guess if your AI works.
Measure it. Evaluate continuously. Ship with confidence.



PROITBRIDGE
Strive For Better Future

AI Coach John | Generative AI Fundamentals

RAG — Give Your LLM Access to Your Data



1 WHY RAG EXISTS

LLMs are frozen in time. They know nothing about your company's internal docs, your product, your customer data, or anything that happened after training. Fine-tuning is expensive and slow.

RAG solves this with retrieval at inference time — no retraining required.

Without RAG: LLM answers from training data only → stale, hallucinated

With RAG: LLM answers from your documents → grounded, current, accurate

2 THE SIMPLE RAG FLOW

Documents
(PDFs, Docs, Web pages, DBs)

↓

Chunking
Split into small pieces
(300–512 tokens)

↓

Embedding Model
Convert each chunk to
a vector [0.12, −0.87, 0.44, ...]

↓

Vector Database
Store all vectors
(Pinecone, Weaviate, pgvector)

↓

User Query
Embed the query →
Search for similar vectors

↓

Retrieval
Return Top-K most
relevant chunks

↓

Context Injection
Stuff chunks into the prompt

↓

LLM
Reads context + generates
grounded answer

↓

User gets accurate,
document-backed response

3 CORE CONCEPTS

A EMBEDDINGS



Dense vector representations of text where semantic similarity = geometric proximity.

"King" and "Queen" are close.
"Dog" and "automobile" are far.

The embedding model converts text → a list of ~1500 numbers.

```
"How do I reset my password?"
→ [0.12, -0.87, 0.44, 0.91, ...]

"Steps to change my password"
→ [0.11, -0.85, 0.46, 0.89, ...]
```

→ These two vectors are very similar
→ retrieved together

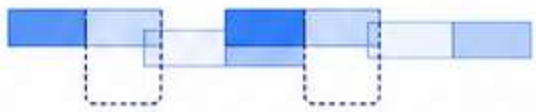
B CHUNKING



Documents must be split into retrievable pieces.

Too large → low precision.
Too small → missing context.

Typical: 256–512 tokens with 50–100 token overlap between chunks.



Overlap preserves context

C VECTOR DATABASE



Stores embeddings and enables fast approximate nearest-neighbor search.

Can query millions of vectors in milliseconds.

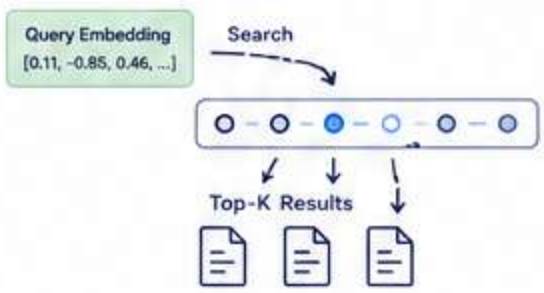
Tool	Best For
Pinecone	Managed cloud, easy setup
Weaviate	Multi-modal, open source
pgvector	Already using PostgreSQL
Qdrant	Performance, open source
ChromaDB	Local development

D RETRIEVAL



Find the Top-K chunks whose embedding is closest to the query embedding.

Cosine similarity is the standard distance metric.



E CONTEXT INJECTION

Retrieved chunks are inserted into the LLM prompt as context:



```
System:   "Answer using only the context below."
Context:  [Chunk 1] ... [Chunk 2] ... [Chunk 3]
User:     "What is the refund policy?"
```

→ LLM Generates Grounded Answer

GOAL

Connect your LLM to your data.
Ground every response. Eliminate guessing.



PROITBRIDGE
Strive For Better Future

AI Coach John | Generative AI Fundamentals

Advanced RAG — Beyond Simple Vector Search

THE UNCOMFORTABLE TRUTH

Naive RAG (embed → retrieve → inject) breaks in production.

Retrieval returns wrong chunks. Queries are ambiguous. Rankings are off. Metadata is ignored. The LLM gets bad context and hallucinates anyway — just with citations.



Basic RAG:
50–60%
retrieval accuracy
(typical naive implementation)

→

Advanced RAG:
80–90%+
with the right enhancements

ADVANCED CONCEPTS

1 HYBRID SEARCH

Combine dense vector search (semantic similarity) with sparse keyword search (BM25).
Neither alone is best.

BM25
Finds exact keyword matches

+

Vector Search
Finds semantic matches

Combining them catches what each misses.

Final score = α · BM25_score + (1−α) · vector_score

α = 0.5 is a common starting point

2 RE-RANKING

After retrieving Top-20 chunks by vector similarity, run a cross-encoder reranker to re-score them with full attention over (query, chunk) pairs.
The cross-encoder is slower but far more accurate than embedding similarity.

Return the re-ranked Top-5 to the LLM.

Step 1 Vector search
→ Top 20 candidates (fast)

↓

Step 2 Reranker
→ Re-score all 20 (slower, accurate)

↓

Step 3 LLM gets
→ Top 5 re-ranked (best context)

3 QUERY EXPANSION

The user's query is often a bad search query. Rewrite or expand it before retrieval.

Original: "refund?"

↓

Expanded:
"What is the refund policy? How do I return a product? What are the eligibility requirements for a refund?"

Retrieve for all three → merge results → better coverage

4 METADATA FILTERING

Vector search alone ignores structure. Metadata filters add hard constraints before or after retrieval.

```
Filter:
{ "year": { "$gte": 2023 },
  "department": "legal" }
```



Only retrieve legal documents from 2023 or newer

Prevents retrieving semantically similar but contextually wrong chunks.

5 EVALUATION FOR RAG

Standard LLM eval isn't enough.
RAG needs component-level evaluation:

Metric	What it Measures
Context Precision	Are retrieved chunks actually relevant?
Context Recall	Were all relevant chunks retrieved?
Faithfulness	Does the answer only use retrieved context?
Answer Relevance	Does the answer address the question?

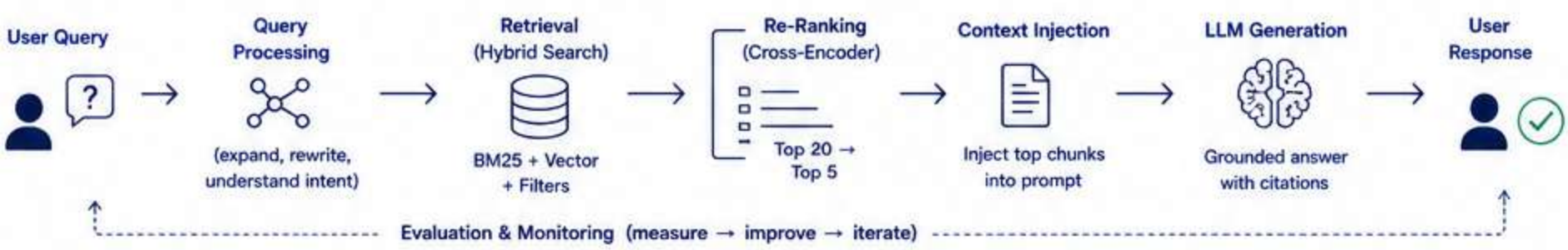
Use RAGAS framework to measure all four automatically.

RAGAS

6 ADVANCED RAG PATTERNS

HyDE	Generate a hypothetical answer → embed it → retrieve similar real docs
Parent-Child	Retrieve small chunks → return their parent chunk for full context
Step-Back	Ask a broader question first → retrieve general then specific
Multi-Query	Generate 3 versions of the query → retrieve for all → merge
FLARE	LLM generates → pauses when uncertain → retrieves → continues
GraphRAG	Build knowledge graph → retrieve via entity relationships

HOW IT ALL WORKS TOGETHER



GOAL

Good RAG is an engineering discipline — not a one-liner. Build it right.



PROITBRIDGE
Strive For Better Future

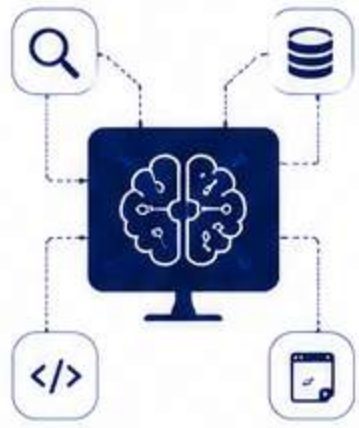
AI Coach John | Generative AI Fundamentals

AI Agents — When the LLM Takes the Wheel



1 WHAT IS AN AGENT?

An agent is an LLM that doesn't just answer — it acts.
It can decide what tool to use, use it, observe the result, decide what to do next, and repeat until the goal is achieved.
The LLM is the brain. Tools are the hands.



Single prompt:
User asks → LLM answers → Done

Agent:
User sets goal → LLM plans → executes tools → observes → re-plans → executes → ... → delivers result

2 CORE CONCEPTS

2.1 TOOL CALLING (FUNCTION CALLING)

The LLM can call external functions — search the web, query a database, run Python, call an API, read a file.
The model outputs a structured JSON tool call.
Your code executes it. The result comes back.
The model continues.

Model output:

```
{
  "tool": "search_web",
  "arguments": { "query": "current NVIDIA
     stock price" }
}
```



2.2 MEMORY

Agents need to remember. Four types:

- **In-context:** The current conversation — limited by context window
- **External (Vector):** Long-term retrieval from a vector DB across sessions
- **Entity Memory:** Track specific facts (user name, preferences, past decisions)
- **Episodic:** Log of what the agent did before (actions + outcomes)

2.3 PLANNING

Breaking a complex goal into executable sub-tasks.

```
Goal: "Write a research report on LLM safety"
Plan:
  1. Search recent papers on LLM safety
     (tool: search_web)
  2. Read and summarize each paper
     (tool: read_url)
  3. Extract key themes across papers
     (LLM reasoning)
  4. Draft report structure (LLM reasoning)
  5. Write each section (LLM generation)
  6. Save final report (tool: write_file)
```

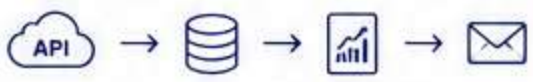
2.4 MULTI-STEP REASONING

Agents use the ReAct loop → Reason → Act → Observe → Repeat.

Thought: "I need to find the CEO of OpenAI first."
↓
Action: search_web("OpenAI CEO 2024")
↓
Observe: "Sam Altman is the CEO of OpenAI."
↓
Thought: "Now I need his recent statements on AI safety."
↓
Action: search_web("Sam Altman AI safety 2024 statements")
↓
...

2.5 WORKFLOW AUTOMATION

Agents can replace multi-step manual workflows:



Pull data from an API → clean it → analyze → send email summary



Monitor logs → detect anomaly → create ticket → notify Slack



Read support email → classify → draft response → route to team

2.6 EVALUATION FOR AGENTS

Agents need evaluation too. Track what matters.

Metric	What it Measures
Task Success Rate	Did the agent complete the task correctly?
Tool Selection Accuracy	Did the agent choose the right tools?
Step Efficiency	Did the agent take the optimal number of steps?
Hallucination Rate	Did the agent generate incorrect information?
Cost per Task	Total tokens + tool costs per completed task
User Satisfaction	Was the user happy with the result?

3 AGENT EXAMPLES

Agent	What it Does	Tools Used	Example Use Case
Research Agent	Searches, reads, synthesizes information from multiple sources	web_search, read_url, write_file, vector_db, summarizer	Build market research reports, literature reviews
Coding Agent	Writes, runs, debugs, fixes, and refactors code	code_interpreter, file_system, linter, git, debugger	Build full-stack features, fix bugs, refactor codebase
Customer Support Agent	Reads tickets, queries knowledge, drafts replies, handles escalations	CRM API, email, knowledge base, search, templates	Answer customer queries, route issues, update CRM
Data Agent	Pulls data, analyzes, creates visualizations and insights	SQL, Python, pandas, plotting libraries, data APIs	Sales reports, KPI dashboards, trend analysis
DevOps Agent	Monitors infrastructure, detects issues, takes corrective action	cloud APIs, monitoring tools, alerting, runbooks, shell	Incident response, auto-scaling, alert triage, deployments

GOAL

Move beyond single prompts.
Build systems that take initiative, use tools, and deliver results.



Production Agent Design — Powerful Agents Need Strong Controls



THE PRODUCTION REALITY

A demo agent that sometimes works is impressive.
A production agent that reliably works for 10,000 users under adversarial conditions — that's engineering.

Single-agent demos are easy.
Production multi-agent systems with real users, real data, and real consequences are an entirely different challenge.





1 MULTI-AGENT SYSTEMS

Decompose complex goals across specialist agents that collaborate. Each agent has a narrow role and defined interface.



Frameworks:
LangGraph · CrewAI · AutoGen · AgentTorch

2 HUMAN-IN-THE-LOOP (HITL)

Not all decisions should be autonomous. High-stakes actions need human approval. Design explicit checkpoints where the agent pauses and asks.

Low risk: Agent sends a Slack message	No approval needed
Medium risk: Agent creates a Jira ticket	Approval optional
High risk: Agent deploys code to production	Requires human sign-off



3 GUARDRAILS

Agents can take real actions with real consequences — sending emails, deleting files, charging customers. Guardrails define what the agent is allowed to do.

- Allowed: read files from /data/reports
- Blocked: delete any file
- Blocked: send emails to external domains
- Blocked: spend more than $50 per task without approval

4 OBSERVABILITY

You must be able to see exactly what every agent did, in what order, with what inputs and outputs.

Trace every agent call:

- Which tool was called?
- What were the arguments?
- What was the result?
- How long did it take?
- What did it cost?
- Did it hit a guardrail?

Tools:
LangSmith · Langfuse · Arize · Helicone

5 FAILURE HANDLING

Agents fail. Tools time out. APIs return errors. LLMs hallucinate a tool argument. Design for failure explicitly.

Failure patterns to design for:

- Tool call returns error → retry with backoff → fallback tool → ask human
- Agent loops without progress → step limit → timeout → alert operator
- LLM produces invalid JSON for tool call → parse error → retry with correction prompt
- Goal is ambiguous → agent asks for clarification before acting

PRODUCTION AGENT CHECKLIST

- [ ] Every tool has input validation and error handling
- [ ] Maximum step limit set (prevent infinite loops)
- [ ] Maximum cost per task limit enforced
- [ ] High-risk actions require human approval
- [ ] Full trace logging for every agent run
- [ ] Agent cannot access data beyond its permission scope
- [ ] Fallback path defined for every critical tool
- [ ] Eval suite running on agent traces post-deploy
- [ ] Alerting on failure rate, cost spikes, loop detection
- [ ] Rollback plan if a bad agent run causes damage

THE FULL GENAI LEARNING PATH — COMPLETED

Step 1	→	LLM Fundamentals	Foundation
Step 2	→	Prompt Engineering	Communication
Step 3	→	Production Thinking	Scale
Step 4	→	Security & Safety	Protection
Step 5	→	Hallucination & Eval	Quality
Step 6	→	RAG Fundamentals	Data Access
Step 7	→	Advanced RAG	Better Retrieval
Step 8	→	AI Agents	Autonomy
Step 9	→	Production Agents	Reliability

FINAL REALITY CHECK

Anyone can prompt an LLM.

Few can build systems that are secure, reliable, and production-grade.

You now have the map.
The rest is execution.

Powerful agents need strong controls.
Build responsibly.